

This work is licensed under a Creative Commons license



Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Number Representation in Computer Memory

Signed|Unsigned Integers and Floating Points

Simone Aonzo, PhD



Positional Binary Recap

- Base-2 positional system
 - a bit at position i has weight 2^i
- Example (unsigned): $11001_2 =$
 - $1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 25_{10}$
- Most Significant Bit (MSB) $\rightarrow$ leftmost bit
 - In signed encodings, it is the sign:
 - $0 \rightarrow +$
 - $1 \rightarrow -$
- Least Significant Bit (LSB) $\rightarrow$ rightmost bit
 - Whether you are using unsigned or two's complement (signed)
 - $0 \rightarrow$ the number is even
 - $1 \rightarrow$ the number is odd

Computers store bit streams...

CPU registers, RAM, HDD, SSD, etc. store sequences of bits, and their usage then depends on how we interpret them $\implies$ a bit pattern could represent a signed/unsigned integer, a character/string, an instruction, ...

Unsigned Integers ($\mathbb{N}$ atural numbers)

- Interpretation: $U = \sum_{i=0}^{n-1} b_i \cdot 2^i$
- Range for n bits: $[0 \text{ to } 2^n - 1]$

Example (8-bit)

Bits	Value
0000 0000 ₂	0 ₁₀
0000 0101 ₂	5 ₁₀
0000 1010 ₂	10 ₁₀
1111 1111 ₂	255 ₁₀

Properties

- Modular arithmetic mod 2^n (wrap-around)
- Left logical shift $\ll 1$ multiplies by 2^n (if no overflow)
- Right logical shift $\gg 1$ divides by 2^n (floor)

Signed Encodings – One's complement

The one's complement $\bar{x}$ of a binary number x (on n -bits) is formed by flipping (turning 1s into 0s and 0s into 1s) every bit. AKA logical negation (**NOT**)

$$\bar{x} \equiv 2^n - x - 1 \pmod{2^n}$$

- The MSB is the sign
- Problem: two zeros! E.g., on 8-bits
 - $+0 = 00000000$
 - $-0 = 11111111$
- This makes arithmetic messy...
- ... and we need dedicated hardware to deal with it

Signed Encodings – Two's complement ($\mathbb{Z}$: integers)

Given an n -bit positive ($\implies MSB = 0$) number x

- 1 Invert all bits (one's complement): $\bar{x}$
- 2 Add 1: $\bar{x} + 1 \pmod{2^n}$

AKA arithmetic negation (**NEG**)

Example (8-bit): $+5 = 0000\ 0101_2$

- 1 Invert $\rightarrow 1111\ 1010_2$
- 2 Add $+1 \rightarrow 1111\ 1011_2 = -5$

Mathematically, the two's complement **2C** (on n -bits) of x :

$$2C(x) \equiv \bar{x} + 1 \equiv 2^n - x - 1 + 1 \equiv 2^n - x \pmod{2^n}$$

Two's Complement – Properties

- MSB acts as a **sign bit**
- Range: -2^{n-1} to $+(2^{n-1} - 1)$
 - Beware: most negative (-2^{n-1}) has no positive counterpart in n bits!
- Unique zero (no -0), contiguous range
- Self-inverse: applying to $2C(-x)$ again returns $x \pmod{2^n}$
- Addition & subtraction with the same hardware!
 - Subtraction $a - b$ as $a + (-b)$ using two's complement of b
 - Math: $(a - b) \pmod{2^n} = (a + (2^n - b)) \pmod{2^n}$

It is indeed the standard signed encoding used in modern computers

Ranges by Bit-Width

Bits	Unsigned Range	Two's Complement Range
4	[0, 15]	[−8, 7]
8	[0, 255]	[−128, 127]
16	[0, 65 535]	[−32 768, 32 767]
32	[0, 4 294 967 295]	[−2 147 483 648, 2 147 483 647]
64	[0, $2^{64} - 1$]	[-2^{63} , $2^{63} - 1$]

Example: Subtraction via Two's Complement

Compute $19_{10} - 27_{10}$ in 8-bit two's complement:

- ① $19 \rightarrow 0001\ 0011_2$
- ② $27 \rightarrow 0001\ 1011_2$
- ③ Two's complement of $27_{10} = 0001\ 1011_2$:
 - invert $\rightarrow 1110\ 0100_2$
 - add 1 $\rightarrow 1110\ 0101_2$
- ④ Add: $0001\ 0011_2 + 1110\ 0101_2 = 1111\ 1000_2$

Interpret $1111\ 1000_2$ as signed. What is its absolute value?

Re-apply the two's complement:

- invert $\rightarrow 0000\ 0111_2$
- add 1 $\rightarrow 0000\ 1000_2$
- $0000\ 1000_2 = 8_{10}$

$\Rightarrow 1111\ 1000_2 = -8_{10}$ (Correct: $19 - 27 = -8$)

Overflow with Examples (8-bit)

Overflow in two's complement happens when adding two numbers with the same sign yields a result with the opposite sign

Positive overflow

$$\begin{array}{r} 0111\ 1111_2\ (127) \\ +\ 0000\ 0001_2\ (1) \\ \hline =\ 1000\ 0000_2 \\ \text{interpreted as } -128 \Rightarrow \text{overflow} \end{array}$$

Negative overflow

$$\begin{array}{r} 1000\ 0000_2\ (-128) \\ +\ 1111\ 1111_2\ (-1) \\ \hline =\ 0111\ 1111_2\ (127) \Rightarrow \text{overflow} \end{array}$$

Sign Extension

When widening from n to $m > n$ bits, preserves numeric value across widths

- Unsigned Extension: pad with zeros on the left
- Signed Extension in Two's complement
 - pad with copies of the sign bit (MSB)

Example (4-bit to 8-bit)

- Unsigned/Signed: $0110_2 (+6) \rightarrow 0000\ 0110_2$
- $1010_2 (-6|10) \rightarrow$
 - Unsigned: $0000\ 1010_2$
 - Signed: $1111\ 1010_2$

Floating-Point ($\mathbb{Q}$: rational numbers)

Representation of *Real numbers* using a **fixed** number of bits.

But, the position of the decimal point is **not fixed** $\implies$ *floating*

IEEE 754 Representation [32 | 64 bits] [single | double precision]

A floating-point number is stored as:

- **Sign** s [1 | 1 bit] — positive or negative
- **Exponent** E [8 | 11 bit] — scales the number
 - The **bias** allows storing negative exponents using only unsigned integers
 - single precision: bias = 127, double precision: bias = 1023
- **Mantissa** [23 | 52 bit] — stores significant digits
 - f : number of mantissa (fraction) bits
 - $b_i \in \{0, 1\}$: i -th fraction bit (i.e., with weight 2^{-i})

$$\text{value} = (-1)^s \cdot 2^{(E-\text{bias})} \cdot \left(1 + \sum_{i=1}^f b_i \cdot \frac{1}{2^i} \right)$$

Example: Computing a Floating-Point Number

$$\text{value} = (-1)^s \cdot 2^{(E-\text{bias})} \cdot \left(1 + \sum_{i=1}^f b_i \cdot \frac{1}{2^i}\right)$$

IEEE 754 single precision (32 bits)

- Sign $s = 0 \implies (-1)^0 = +1$
- Exponent $E = 10000001_2 = 129_{10} \implies 2^{129-127} = 2^2 = 4_{10}$
- Mantissa = $01000000000000000000000_2$
 - only one bit in position $i = 2$
 - $1 + \frac{1}{2^2} = 1 + 0.25 = 1.25_{10}$

$$\text{value}_{10} = +1 \cdot 4 \cdot 1.25 = \boxed{5.0}$$

IEEE 754 Arithmetic: **use with caution!**



- **Logarithmically spaced:** precision scales with magnitude
- **Catastrophic cancellation**
Subtracting nearly equal numbers destroys significant digits
- **Special values exist**
 $+\infty$, $-\infty$, NaN, subnormals, signed zero ($+0$ and -0)
- **Rounding modes matter**
IEEE 754 defines multiple rounding behaviors
- **Not algebraically sound:** associativity and distributivity may fail
- **Equality comparison is dangerous**
Do not use “if ($x == y$)”; use tolerances: $|x - y| < \varepsilon$

Takeaway: *a messed up scientific notation in base 2*

Finite, discrete, deterministic, but only approximates real arithmetic

For an in-depth study, look at the Wikipedia pages [Wikb] [Wika]

[Wika] [Wikipedia](#).

IEEE 754.

https://en.wikipedia.org/wiki/IEEE_754.

[Wikb] [Wikipedia](#).

Two's complement.

https://en.wikipedia.org/wiki/Two's_complement.